

H3Match: A Hybrid Heterogeneous Hypergraph Matching Method for Subcircuit Identification

Bohao Li¹, Qingsong Peng¹, Changhong Wang², Tianming Ni³, Tinghuan Chen⁴, Qi Sun^{1,*}, Cheng Zhuo^{1,*}

¹Zhejiang University,

²Shandong Yunhai Guochuang Cloud Computing Equipment Industry Innovation Co. Ltd.,

³Anhui Polytechnic University, ⁴The Chinese University of Hong Kong, Shenzhen

*Corresponding authors, email: {qisunchn, czhuo}@zju.edu.cn

Abstract—Subcircuit matching is widely applied in logic synthesis, design verification, hardware security, *etc.* Previous works employ redundant circuit representations, coupled with time-consuming enumerative search methods. Subsequent works use a hybrid “approximate filtering – exact verification” framework, but the numerous false negatives predicted by the graph neural network (GNN) based filtering lead to severe matching failure. In this paper, an improved hybrid method named H3Match is proposed to achieve a better tradeoff between runtime, accuracy, and false negative rate. First, we model the circuits as hypergraphs to fully capture the topology and construct diverse heterogeneous hyperedge features to facilitate the learning of circuit topologies. Second, to reduce the false negatives, we reformulate the subgraph matching problem as matching directed acyclic graphs (DAGs) with embedded circular structure information and develop a directed GNN-based *approximate* matching approach to identify potential matching subcircuits. Finally, we propose a general mixed integer nonlinear programming (MINLP) formulation for *exact* verification, with convergency speed accelerated by extracting the initial solution from the results in *approximate* matching. Experimental results show that our *approximate* method outperforms state-of-the-art (SOTA) methods by 4.31% in accuracy while achieving virtually zero false negatives. Our exact verification is on average 4.16× faster than SOTA *exact* methods. Overall, the end-to-end flow achieves a 7.08× speedup compared to existing approaches.

I. INTRODUCTION

Subcircuit matching involves identifying subcircuits in a target circuit that have the same device attributes and connectivity as the query circuit. It is a fundamental step in the EDA flow, with wide applications in layout generation, design verification, reverse engineering, *etc.* [1]–[3]. The current matching methodology primarily comprises two steps: representing the circuit in a graph-based form and performing subgraph isomorphism on the resulting graphs. Due to the NP-complete nature of subgraph isomorphism, the matching time increases significantly as integrated circuits continue to scale up. Therefore, developing efficient matching algorithms is imperative to accelerate the circuit design flow.

Previous studies mainly focus on developing subgraph isomorphism algorithms, with insufficient emphasis on circuit modeling methods. Typically, these studies represent circuits as graphs [4], [5], bipartite graphs [6]–[8], or heterogeneous multigraphs [2], [9]. Since edges in these graph structures can only represent pairwise relationships, the one-to-many con-

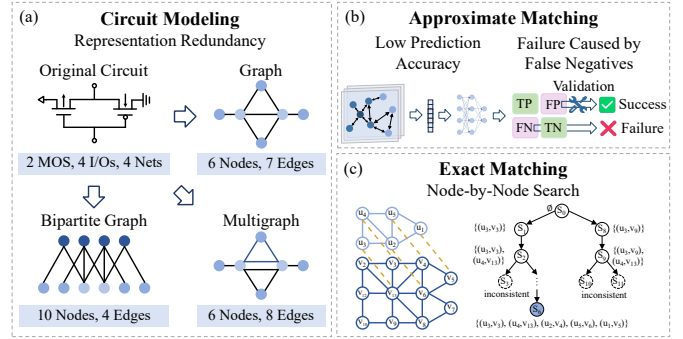


Fig. 1 Major challenges faced by existing hybrid matching methods. (a) More nodes or edges are required for circuit modeling. (b) Approximate matching suffers from low accuracy and numerous false negatives that can lead to matching failure. (c) Node-by-node search is time-consuming.

nections between nets and devices are decomposed into multiple one-to-one connections. Consequently, additional edges (as in graphs or multigraphs) or nodes (as in bipartite graphs) are required to maintain the original structure, as shown in Fig. 1(a). Therefore, these modeling methods face two major issues. First, these approaches introduce redundancy in representation, which increases the runtime of subcircuit matching. Second, due to the decomposition process, the rich semantic features inherent in the connections between nets and devices are lost, limiting subsequent matching steps from effectively leveraging this information to enhance efficiency.

For the matching algorithms, prior works primarily employ search-based methods for exact subcircuit matching. Some arts assign discriminative labels to nodes based on device attributes or neighborhood structures, and establish mappings between nodes via breath first search or depth first search [10], [11]. Other works adopt tree-search methods originated from subgraph isomorphism research [12]–[14]. However, when applied to large-scale circuits, developing discriminative labeling algorithms is highly challenging, and the exhaustive tree-search become time-consuming.

With advancements in deep learning, GNNs have emerged as a promising solution [15]–[18]. By extracting graph topology into embedding vectors and using multilayer perceptron

(MLP) for prediction, GNN-based methods eliminate the need for node-wise matching and achieve a speedup of over $10 \times$ compared to conventional methods. However, these methods are unable to provide the one-to-one correspondences between the node set of two graphs and suffer from low accuracy due to information loss in message-passing. Therefore, recent works adopt a hybrid method, where GNN is first used to filter out candidate matches, followed by verification using search-based methods [19], [20]. This approach achieves a tradeoff between runtime and accuracy, making it a promising solution. Nonetheless, two critical challenges persist: first, the search-based verification dominates the overall runtime, posing a bottleneck for further acceleration. Second, in most applications, the number of positive examples is significantly smaller than that of negative examples. Therefore, the search-based method only verifies the positive results from approximate matching. As a result, only false positives (FPs) can be corrected, leaving false negatives (FNs) unaddressed, which leads to the failure of the entire matching flow. Therefore, minimizing the number of false negative results in approximate matching is essential, while previous studies overlook this issue.

In this paper, to address the aforementioned challenges, we propose a heterogeneous hypergraph representation for circuit modeling and a hybrid subgraph isomorphism method for efficient subcircuit matching. Our contributions are as follows:

- We introduce an effective heterogeneous hypergraph representation for circuit modeling to reduce representation redundancy and improve accuracy.
- To minimize the false negatives, we reformulate the subgraph matching problem as sub-DAG matching and develop a directed HGNN-based approximate matching method to filter out potential matching subcircuits.
- To enhance the matching speed, we formulate the exact verification as a mixed integer nonlinear programming problem and accelerate the optimization by providing an initial solution extracted from the filtered results.

Experimental results demonstrate that our approximate filtering achieves 98.96% accuracy and nearly 100% recall, while our exact verification is $4.16 \times$ faster than existing approaches. Consequently, our end-to-end flow attains 100% accuracy and a $7.08 \times$ speedup in the subcircuit matching task.

II. PRELIMINARIES

A. Problem Formulation

Definition 1 (Heterogeneous Hypergraph). A heterogeneous hypergraph can be represented as $\mathcal{HHG} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e, m_v, m_e)$, where $\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e$ is the set of nodes, hyperedges, node types and hyperedge types, respectively. m_v and m_e are mapping functions that associate each node in $\mathcal{V}$ or hyperedge in $\mathcal{E}$ with one type in $\mathcal{T}_v$ or $\mathcal{T}_e$. Each hyperedge $e \in \mathcal{E}$ stands for the connection between a group of nodes $\{v_1, v_2, \dots, v_n\}$ where $v_1, v_2, \dots, v_n \in \mathcal{V}$.

Definition 2 (Subgraph Isomorphism). Given a target graph $\mathcal{HHG}_1 = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e, m_v, m_e)$ and a query graph $\mathcal{HHG}_2 = (\mathcal{V}', \mathcal{E}', \mathcal{T}'_v, \mathcal{T}'_e, m'_v, m'_e)$, subgraph isomorphism is to find all

injective functions $f : \mathcal{V}' \rightarrow \mathcal{V}$ such that $\forall u \in \mathcal{V}' : m'_v(u) = m_v(f(u))$; $\forall \{u_1, u_2, \dots, u_n\} \in \mathcal{V}' : (u_1, u_2, \dots, u_n) \in \mathcal{E}' \implies (f(u_1), f(u_2), \dots, f(u_n)) \in \mathcal{E}, m_e((u_1, u_2, \dots, u_n)) = m_e(f(u_1), f(u_2), \dots, f(u_n))$.

Problem 1 (Subcircuit Identification). Given a target circuit and a query circuit, subcircuit identification is to find all subcircuits in the target circuit that have the same topology and device attributes as the query circuit.

B. Hypergraph Neural Network

Graphs serve as an effective model for capturing pairwise relationships between two entities. However, in real-world scenarios, it is often necessary to represent more complex relationships among multiple entities. Therefore, hypergraphs and hypergraph neural networks have been introduced to model and extract features from such relationships. When extending GNNs to hypergraphs, the message-passing framework maintains, while the change from the adjacency matrix to the incidence matrix requires normalization using the node degree matrix $\mathbf{D}_v$ and the hyperedge degree matrix $\mathbf{D}_e$ simultaneously [21], as illustrated in the Equation (1):

$$\mathbf{X}^{(l+1)} = \sigma(\mathbf{D}_v^{-1} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top \mathbf{X}^{(l)}), \quad (1)$$

where $\mathbf{X}^{(l)}$, $\mathbf{H}$, $\mathbf{W}^{(l)}$ represent the node embedding matrix, the incidence matrix, and the weight matrix at the l th layer, respectively. This process can be regarded as first aggregating the features of nodes connected to each hyperedge onto the hyperedge itself, and then aggregating the features of multiple hyperedges onto the central node.

C. Subgraph Isomorphism

Subgraph isomorphism, also known as subgraph matching, is a fundamental problem in graph data analysis. The mainstream research is centered on search-based algorithms [22]–[25], which guarantee to find all matches. VF2 [23] is one of the most commonly used exact algorithms. It determines node matching using subgraph constraints and heuristic rules, then explores other nodes by depth-first search and backtracking. Subsequent studies optimize heuristic rules to enhance the efficiency of search space pruning [24], [26]. However, due to the enormous search space and the exhaustive search strategy, matching on large-scale graphs remains time-consuming.

Another approach represents the node mapping between two graphs as an assignment matrix $\mathbf{S}$, where $\mathbf{S}_{i,j}$ represents a match between the i th node of the target graph and the j th node of the query graph, thus reformulating subgraph isomorphism as an integer programming problem, as shown in Equation (2):

$$\begin{aligned} \min \quad & \|\mathbf{S} \mathbf{A}' \mathbf{S} - \mathbf{A}\|_F^2, \\ \text{s.t.} \quad & \sum_i \mathbf{S}_{i,j} = 1, \sum_j \mathbf{S}_{i,j} \leq 1, \mathbf{S}_{i,j} \in \{0, 1\}, \end{aligned} \quad (2)$$

where $\mathbf{A}'$ and $\mathbf{A}$ denote the adjacency matrices of the target graph and query graph, respectively. Being a nonconvex

MINLP, this optimization problem poses challenges for various approximation techniques, which may fail to yield an exact solution or get stuck in local minima.

The advancements of GNNs for effectively learning low-dimensional representations of graphs facilitate a variety of downstream tasks, including subgraph isomorphism [27], [28]. By leveraging GNNs to extract global graph features and the efficiency of GPU computation, GNN-based methods achieve significantly higher speeds than other approaches, but face serious challenges in accuracy and generalization.

III. H3MATCH

A. Overall Flow

To achieve better overall performance in terms of runtime, FNs, and accuracy, we develop a novel subcircuit matching method. Our method takes two circuits as input: a target circuit and a query circuit. The objective is to identify all subcircuits in the target circuit that match the query circuit.

The overall flow of our framework is outlined as follows: we first employ a netlist parser to extract the topology and device attributes from the input netlists, followed by constructing the heterogeneous hypergraph representation. Subsequently, the heterogeneous hypergraph is converted into a DAG with circular structure information embedded. We use a directed HGNN to embed the DAG and identify potentially isomorphic subcircuits by partial order comparison. Thereafter, we employ an exact subgraph isomorphism approach on the identified small-scale subcircuits to verify the matching results. The overall flow is shown in Fig. 2. Henceforth, subgraphs of the target graph will be referred to as sub-targets.

B. Circuit Modeling

In circuit modeling, we aim to represent the circuit with fewer nodes and edges while fully preserving its topology. Many existing works in the literature model the transistor as four separate nodes, *i.e.*, source, drain, gate, body, and each assigned a unique node feature. Moreover, prior works either decompose nets into multiple pairwise connections between devices or introduce additional nodes to represent the one-to-many relationship between nets and devices.

In our representation, NMOS, PMOS, V_{ss} , V_{dd} , and other devices are each modeled as one node and assigned a 5-dimensional one-hot vector to encode their types. Nets are modeled as heterogeneous hyperedges, with each hyperedge assigned a 9-dimensional feature vector to represent the connections to different device pins. Each element of the vector corresponds to the count of connections to the gate, drain, and source of NMOS and PMOS, as well as the connections to V_{ss} , V_{dd} , and other devices.

Our heterogeneous hypergraph representation takes two advantages. First, fewer nodes and edges are required to represent the circuit, which reduces the structural complexity (compared to graphs) and the number of GNN layers required for learning (compared to bipartite graphs), thereby enhancing the accuracy for approximate matching while shortening the runtime for exact matching. Second, the diverse hyperedge

features derived from heterogeneous hyperedges are fully preserved, providing significant benefits in differentiating subcircuits with similar topologies.

C. Approximate Matching

In approximate matching, we focus on two key aspects. On the one hand, the false negatives should be as low as possible to reduce the overall matching errors within the framework. On the other hand, false positives should be kept at a low level to minimize the computational overhead of subsequent exact matching. Additionally, the matching process needs to be optimized to achieve a matching speed far exceeding that of exact matching. To achieve these objectives simultaneously, we propose the following matching method.

1) *Subgraph Matching Simplification*: We first simplify the subgraph matching problem to a sub-DAG matching for two main reasons. First, the presence of numerous circular structures in transistor-level circuits may cause information confusion during message passing. Second, with circular structures removed, sub-DAG matching becomes a necessary condition for subgraph matching, *i.e.*, sub-DAG matching relaxes the criteria of subgraph isomorphism, which in turn helps reduce false negatives. Therefore, we simplify the heterogeneous hypergraph into a DAG with heterogeneous hyperedges, as shown in Fig. 2.

To minimize the depth of the DAG and thereby reduce the GNN layers required for embedding, we initiate the DAG transformation by determining the root node of the query graph. The eccentricity of each node is calculated as the maximum of the shortest path distances to all other nodes. The node with the smallest eccentricity is selected as the root node. We then add the n -th hop neighbor nodes of the root node to the n -th layer of the DAG. Hyperedges connect nodes between two adjacent layers, with the node in the n -th layer acting as the head node and the nodes in the $(n+1)$ -th layer as the tail nodes. We ensure that each hyperedge connects only one head node. If multiple head nodes are present, the hyperedge is decomposed into several hyperedges, each sharing identical tail nodes but distinct head nodes. Furthermore, removing circular structures may result in a decrease in accuracy. To mitigate this issue, we introduce a new child node to the node with the minimum hop count on the circle, assigning it the average feature value of all nodes within the circle.

2) *Graph Learning Model*: Previous heterogeneous GNNs leverage attention mechanisms to capture various relationships among entities. However, our topology extraction task focuses on distinguishing different hyperedge types. To this end, we assign a learnable parameter to each type of hyperedge for computational simplicity. Consequently, the incidence matrix can be formulated as Equation (3):

$$\mathbf{H} = \sum_{r \in \mathcal{T}_e} w_r \mathbf{H}_r. \quad (3)$$

To learn the low-dimensional representation of the heterogeneous DAG, we extend the HGNN model from Equation (1)

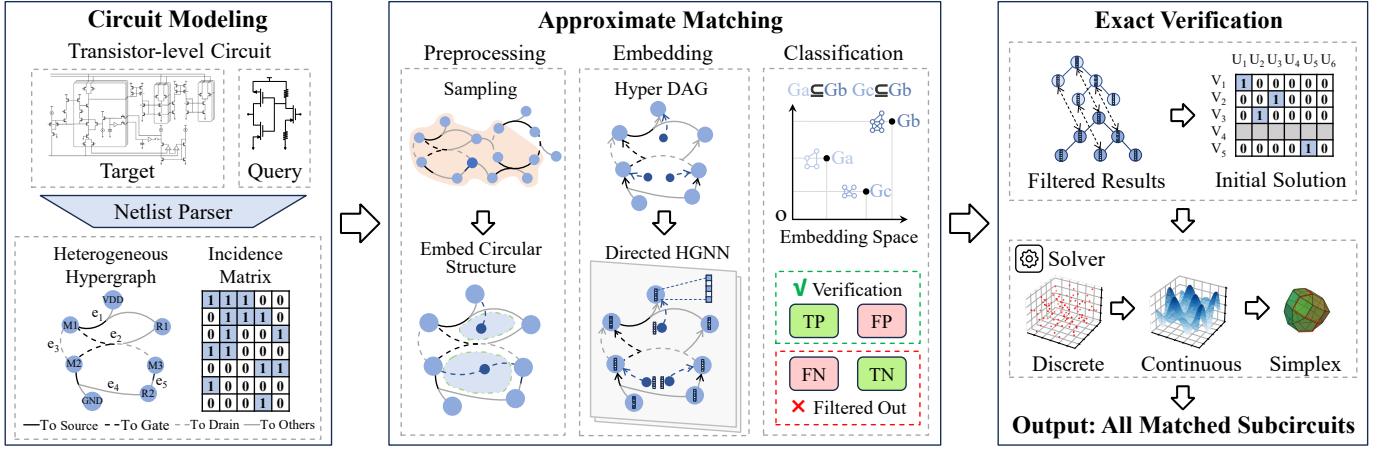


Fig. 2 Overview of the proposed method.

to directed hypergraphs. The directed HGNN maintains a two-stage message passing process. In the first stage, messages are passed from tail nodes to hyperedges, while in the second stage, messages are passed from hyperedges to head nodes. By employing the directed HGNN model, the heterogeneous topological information is aggregated at the root node of the DAG. The embedding of the root node is then used to predict subgraph relationships.

Inspired by NeuroMatch [27], we introduce partial order comparison to predict subgraph relationships based on the embeddings. Given that both subgraph relationships and dominance relationships between graph embeddings exhibit partial order properties, we aim for the GNN model to preserve this partial order relationship during graph embedding. This enables subgraph isomorphism to be simplified to comparing the magnitudes of embedding vectors, resulting in a tenfold speed improvement over traditional MLP-based predictions. To achieve the above objective, a max-margin loss is utilized, as shown in Equation (4):

$$L(\mathbf{z}_q, \mathbf{z}_t) = \alpha \sum_{(\mathbf{z}_q, \mathbf{z}_t) \in \mathcal{P}} \|\max\{0, \mathbf{z}_q - \mathbf{z}_t\}\|_2^2 + \sum_{(\mathbf{z}_q, \mathbf{z}_t) \in \mathcal{N}} \max\{0, \beta - \|\max\{0, \mathbf{z}_q - \mathbf{z}_t\}\|_2\}. \quad (4)$$

For positive examples, we expect the sub-target embedding $\mathbf{z}_t$ to dominate the query embedding $\mathbf{z}_q$, i.e., $\mathbf{z}_q$ should be positioned in the lower-left of $\mathbf{z}_t$ in the embedding space. For negative examples, we require $\mathbf{z}_q$ and $\mathbf{z}_t$ to violate the dominance relationship and maintain a distance of at least β in the embedding space. Given that positive examples are scarce in our subcircuit identification task and the cost of misclassifying positive examples is significant, we assign a higher priority to the loss of positive examples by introducing a weight α in the loss function.

3) *Sub-DAG Matching*: Benefiting from our heterogeneous DAG representation and graph learning model, we utilize the following three techniques to further improve performance.

First, following the root node selection method described before, we determine the root node of the query graph. We then sample neighborhoods from the target graph, centered at each node, with the same radius as the query graph. In comparison, prior research uses a sampling radius twice that of the query graph to ensure the sub-target covers the query. As a result, our method only uses a $2k$ -layer GNN to embed a k -hop query graph, while other approaches twice the number of layers, which not only increases matching time but also risks over-smoothing.

Second, to reduce the number of matching executions, we propose two filtering criteria aimed at excluding non-isomorphic sub-targets. First, only two nodes with identical features are allowed to check the subgraph relationship. Second, after converting the neighborhoods of the two nodes into DAGs, the number of nodes at each layer of the DAG is counted. Subsequent matching is performed only if each layer of the sub-target DAG contains at least as many nodes as the corresponding layer in the query DAG.

Third, to fully exploit the diverse hyperedge features, we aim to conduct message-passing based on these features. Therefore, we apply a duality transformation to the hypergraph, i.e., swapping the roles of nodes and hyperedges. In the transformation, the topology is fully preserved, and the hyperedge features become the node features in the dual hypergraph.

To summarize, in the sub-DAG matching flow, we first identify the root node of the query graph and sample the neighborhood of each node in the target graph with the same radius as the query graph. Next, we filter out non-isomorphic sub-targets based on the two criteria. Subsequently, we perform dual transformation and DAG conversion to the query and sub-target using the aforementioned method. Finally, we extract topological features using our directed HGNN model and make predictions through partial order comparison.

D. Exact Matching

To improve the matching accuracy, we employ an exact verification method to validate the examples classified as

matched in approximate matching. The conventional subgraph matching formulation is extended to accommodate the matching of heterogeneous hypergraphs, as shown in Equation (5):

$$\begin{aligned}
\min \quad & \left\| \mathbf{P}^{(v)} \mathbf{H}' \mathbf{P}^{(e)} - \mathbf{H} \right\|_F^2 + \left\| \mathbf{P}^{(v)} \mathbf{F}' - \mathbf{F} \right\|_F^2 \\
& + \left\| (\mathbf{P}^{(e)})^\top \mathbf{K}' - \mathbf{K} \right\|_F^2 \\
\text{s.t.} \quad & \mathbf{P}^{(e)} = \{0, 1\}^{|E'| \times |E|}, \mathbf{P}^{(v)} = \{0, 1\}^{|V| \times |V'|}, \\
& \sum_j p_{i,j}^{(e)} \leq 1, \sum_i p_{i,j}^{(e)} = 1, \sum_j p_{i,j}^{(v)} = 1, \sum_i p_{i,j}^{(v)} \leq 1,
\end{aligned} \tag{5}$$

where $\mathbf{H}'$ and $\mathbf{H}$ represent the incidence matrix of the target graph and the query graph. $\mathbf{F}$, $\mathbf{K}$ represent the node feature matrix and hyperedge feature matrix. The assignment matrices $\mathbf{P}^{(v)}$ and $\mathbf{P}^{(e)}$ are used to permute and select rows and columns of $\mathbf{H}'$, which corresponds to reordering the nodes and hyperedges of the sub-target graph, followed by selecting a subset to match with the query graph. The objective function consists of three terms that represent the matching of the input graphs' topology, node features, and hyperedge features, respectively. If $\mathbf{P}^{(v)}$ and $\mathbf{P}^{(e)}$ exist such that the objective function equals zero, it indicates that the two input graphs are subgraph isomorphic. Given that the formulation is a nonconvex MINLP problem with numerous variables and constraints, we adopt the following heuristics and settings to accelerate the solution.

First, to reduce the number of variables, we prune all hyperedges in the sub-target whose features do not exist in the query graph and further remove the isolated nodes resulting from pruning. Notably, while this approach still holds for graphs and bipartite graphs, the rich hyperedge features enable our method to prune significantly more hyperedges.

Second, to accelerate the convergency, we extract prior knowledge from approximate matching to provide an initial solution for the solver. The approximate matching outputs a pair of potential matching DAGs, where the two root nodes are assumed to be matched. Additionally, we select nodes with unique features from each layer in each DAG and match nodes with identical features within the same layer between both DAGs. Therefore, the corresponding rows in $\mathbf{P}^{(v)}$ for these matched nodes are set to 0 or 1. Notably, the approximate matching may provide incorrect node mappings for positive examples (e.g., the root nodes do not match) or produce node mappings for negative examples. Our method still guarantees an exact solution without an obvious change in matching time.

Third, in the sampling stage of approximate matching, the radius of the sub-target is set equal to that of the query, which ensures that there is at most one match between them. Owing to this, we execute a subgraph isomorphism check, i.e., terminating once a minimum value is found, rather than determining all minimum values of the nonconvex problem. Based on the above heuristics, the formulation can be effectively solved using relaxation and the simplex method.

TABLE I Statistics of the Testset.

	# Transistor	# Net		# Transistor	# Net
Case1	26520	29178	Case5	60775	51655
Case2	45187	41553	Case6	104892	98716
Case3	47561	47787	Case7	119927	112271
Case4	60110	64670	Case8	190562	180316

TABLE II Performance of our approximate matching method against **SOTA** methods on the validation set.

Query Size	NeuroMatch Acc.	# FN	MLCAD'24 Acc.	# FN	D2Match Acc.	# FN	Ours Acc.	# FN
25	88.50%	155	93.66%	17	94.90%	365	99.31%	0
50	87.37%	129	91.90%	49	94.43%	424	99.14%	0
75	81.93%	276	91.08%	55	94.85%	435	98.86%	0
100	80.46%	328	89.80%	53	94.40%	610	98.51%	0

TABLE III Runtime comparison between our methods and **SOTA** methods. Runtime: seconds.

Query Size	Approximate Methods				Exact Methods	
	NeuroMatch	MLCAD'24	D2Match	Ours	VF3	Ours
25	0.009	0.017	0.047	0.009	0.086	0.012
50	0.013	0.022	0.069	0.012	0.794	0.145
75	0.015	0.025	0.076	0.015	1.941	0.784
100	0.018	0.034	0.160	0.020	3.661	2.418

* The VF3 algorithm refers to the exact matching step in MLCAD'24, rather than the VF3 baseline.

IV. EXPERIMENT

A. Experimental Setup and Dataset

We conduct subcircuit identification on eight in-house transistor-level circuits, as summarized in TABLE I. We first extract the topology and device types using the netlist parser, followed by constructing the graph and hypergraph representations with the NetworkX [29] and HyperNetX [30] library. We implement our method on a Linux server with Intel Xeon 8475B CPUs and NVIDIA RTX 4090 GPUs.

In approximate matching, we train the models on a randomly generated dataset consisting of 60,000 examples. Validation is performed on 20,000 examples obtained by extracting the neighborhoods of randomly selected nodes from the 8 test cases. To improve the accuracy of positive examples, the ratio of positive to negative samples is set to 1:1 for both the training and validation sets. The weight parameter α and margin β in the loss function are 2 and 0.1, respectively. For all graph learning methods, the batch size is set to 64.

In the exact matching stage, our method and MLCAD'24 execute a subgraph isomorphism check on each pair of the query graph and the sub-target graph. All exact matching experiments are repeated 100 times, and the final results are determined after discarding the top 5% and bottom 5% of values and then calculating the average.

In the comparison of end-to-end methods, the VF3 baseline performs a complete subgraph matching to identify all isomorphic subgraphs within the entire target graph. The runtime of hybrid methods comprises four parts: circuit modeling, neighborhood extraction, GNN inference, and exact matching, whereas the runtime of the search-based method is composed of circuit modeling and exact matching.

TABLE IV The performance comparisons of our method against other end-to-end approaches. Runtime: seconds.

Case	Query Size	VF3		MLCAD'24		Ours		Case	Query Size	VF3		MLCAD'24		Ours	
		Acc.	Runtime	Acc.	Runtime	Acc.	Runtime			Acc.	Runtime	Acc.	Runtime	Acc.	Runtime
1	25	100.00%	37.25	99.71%	27.82	100.00%	3.04	5	25	100.00%	61.45	99.87%	63.67	100.00%	6.90
	50	100.00%	66.69	99.49%	147.79	100.00%	6.57		50	100.00%	155.47	99.54%	338.61	100.00%	14.89
	75	100.00%	101.57	99.43%	363.87	100.00%	23.75		75	100.00%	236.36	99.55%	833.78	100.00%	54.20
	100	100.00%	154.34	99.57%	718.56	100.00%	75.55		100	100.00%	327.23	99.61%	1646.62	100.00%	172.84
2	25	100.00%	55.54	99.74%	47.26	100.00%	5.05	6	25	100.00%	125.26	99.72%	109.98	100.00%	12.01
	50	100.00%	111.32	99.56%	251.68	100.00%	10.92		50	100.00%	278.20	99.69%	584.50	100.00%	25.90
	75	100.00%	179.95	99.46%	619.85	100.00%	40.06		75	100.00%	499.85	99.41%	1439.13	100.00%	93.83
	100	100.00%	269.45	99.59%	1224.20	100.00%	128.19		100	100.00%	582.50	99.48%	2842.01	100.00%	298.68
3	25	100.00%	59.13	99.80%	49.72	100.00%	5.29	7	25	100.00%	150.22	99.85%	125.55	100.00%	13.53
	50	100.00%	111.38	99.50%	264.88	100.00%	11.44		50	100.00%	344.27	99.58%	668.08	100.00%	29.21
	75	100.00%	196.26	99.51%	652.39	100.00%	42.09		75	100.00%	478.65	99.48%	1645.21	100.00%	106.68
	100	100.00%	241.84	99.49%	1288.49	100.00%	134.82		100	100.00%	663.30	99.50%	3249.17	100.00%	340.69
4	25	100.00%	73.65	99.77%	63.07	100.00%	6.93	8	25	100.00%	258.15	99.76%	199.60	100.00%	21.60
	50	100.00%	162.45	99.50%	335.00	100.00%	14.93		50	100.00%	547.18	99.59%	1061.67	100.00%	46.62
	75	100.00%	241.88	99.46%	824.76	100.00%	53.91		75	100.00%	911.18	99.61%	2614.32	100.00%	169.83
	100	100.00%	335.59	99.41%	1628.70	100.00%	171.35		100	100.00%	1249.65	99.53%	5162.99	100.00%	541.77

B. Baselines

We select the following SOTA methods from each of the three subgraph matching methods, *i.e.*, search-based, GNN-based, and hybrid, as our baselines:

- VF3 [24]: Search-based method with tree-search and backtracking framework. The SOTA method for sparse graph matching [31]. Our experiments are performed using the Python interface of VF3 [32].
- NeuroMatch [27]: GNN-based method with partial order learning.
- D2Match [28]: GNN-based method with subtree matching degeneracy.
- MLCAD'24 [19]: Hybrid approach with HGNN-based filtering and VF3-based verification.

As previous methods are only capable of handling graph structures, we convert the netlists into graph representations via clique expansion. For the graph learning baselines, we adopt the hyperparameters provided in the original studies to ensure consistency.

C. Comparisons against SOTA Methods

We evaluate the performance of our graph learning method against SOTA methods on the validation set, with the accuracy and number of FN listed in TABLE II. Our method maintains an accuracy of over 98.5% across different sizes of query graphs, outperforming other baselines by an average of at least 4.31%. On the validation set, our method maintains zero false negatives, demonstrating the effectiveness of simplifying subgraph matching to sub-DAG matching.

TABLE III provides a runtime comparison between our methods and SOTA baselines, focusing on the average time of matching a positive and a negative example. Due to the two-stage message-passing process, HGNN-based methods, *e.g.*, MLCAD'24, require longer matching time compared to GNN-based methods NeuroMatch. However, in our method, the efficient subgraph sampling strategy reduces the size of the sub-target, which in turn minimizes the required HGNN layers

and shortens the matching time. Consequently, our method slightly outperforms NeuroMatch on small-scale graphs and remains competitive on query graphs with 100 nodes. In terms of exact matching, we conduct the isomorphism check using our method and VF3. Our method consistently exceeds VF3 for query graphs of all sizes, achieving an average acceleration of $4.16\times$. In our method, by extracting the prior knowledge from approximate matching, up to 80% of the nodes in the query graph can establish an initial solution, thereby improving the convergence speed by at least twofold.

Due to the limited accuracy of GNN-based methods, which restricts their application in circuit identification, we compare our approach against existing search-based and hybrid methods, as illustrated in TABLE IV. For the eight test cases, our method demonstrates significant improvements in runtime compared to existing methods. Specifically, for query graphs of 25, 50, 75, and 100 nodes, our method is on average $10.93\times$, $10.71\times$, $4.68\times$, $2.00\times$ faster than VF3, and $9.24\times$, $22.76\times$, $15.39\times$, $9.53\times$ faster than MLCAD'24. Meanwhile, our method exhibits zero false negatives on the test cases, resulting in an accuracy of 100%.

V. CONCLUSION

In this work, we present H3Match, a novel subgraph isomorphism approach for subcircuit identification. We propose a heterogeneous hypergraph representation for transistor-level circuits, simplify the subgraph matching to sub-DAG matching, and design a hybrid approach with HGNN-based filtering and MINLP-based verification. Compared to prior works, H3Match significantly outperforms the SOTA methods in terms of runtime, accuracy, and FNs.

ACKNOWLEDGMENT

This project was supported by Zhejiang Provincial NSF (Grant No. D24F040002), NSFC (Grant No. W2412034), and Scientific Research Fund of Zhejiang Provincial Education Department (Grant No. Y202457026).

REFERENCES

- [1] P.-H. Wu, M. P.-H. Lin, T.-C. Chen, C.-F. Yeh, X. Li, and T.-Y. Ho, "A novel analog physical synthesis methodology integrating existent design expertise," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 2, pp. 199–212, 2014.
- [2] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. K. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar, "Gnn-based hierarchical annotation for analog circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [3] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu, "Gnn-re: Graph neural networks for reverse engineering of gate-level netlists," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2435–2448, 2021.
- [4] X. Gao, C. Deng, M. Liu, Z. Zhang, D. Z. Pan, and Y. Lin, "Layout symmetry annotation for analog circuits with graph neural networks," in *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021.
- [5] M. Liu, W. Li, K. Zhu, B. Xu, Y. Lin, L. Shen, X. Tang, N. Sun, and D. Z. Pan, "S 3 det: Detecting system symmetry constraints for analog circuits with graph similarity," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2020.
- [6] H.-Y. Su, C.-H. Hsu, and Y.-L. Li, "Subhunter: A high-performance and scalable sub-circuit recognition method with prüfer-encoding," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015.
- [7] D. Cheremisinov and L. Cheremisinova, "Subcircuit pattern recognition in transistor level circuits," *Pattern Recognition and Image Analysis*, vol. 30, pp. 160–169, 2020.
- [8] N. Rubanov, "Subislands: The probabilistic match assignment algorithm for subcircuit recognition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 1, pp. 26–38, 2003.
- [9] T. Chen, Q. Sun, C. Zhan, C. Liu, H. Yu, and B. Yu, "Analog ic aging-induced degradation estimation via heterogeneous graph convolutional networks," in *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021.
- [10] M. Ohlrich, C. Ebeling, E. Ginting, and L. Sather, "Subgemini: Identifying subcircuits using a fast subgraph isomorphism algorithm," in *Proceedings of the 30th International Design Automation Conference*, 1993.
- [11] W. Kim and H. Shin, "Hierarchical lvs based on hierarchy rebuilding," in *Proceedings of 1998 Asia and South Pacific Design Automation Conference*, 1998.
- [12] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar, "Gana: Graph convolutional network based automated netlist annotation for analog circuits," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2020.
- [13] R. S. Rajarathnam, Y. Lin, Y. Jin, and D. Z. Pan, "Regds: A reverse engineering framework from gdsii to gate-level netlist," in *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2020.
- [14] Z. Wu, I. Song, and I. Savidis, "Hybrid utilization of subgraph isomorphism and relational graph convolutional networks for analog functional grouping annotation," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023.
- [15] Q. Xu, H. Geng, S. Chen, B. Yuan, C. Zhuo, Y. Kang, and X. Wen, "Goodfloorplan: Graph convolutional network and reinforcement learning-based floorplanning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 10, pp. 3492–3502, 2021.
- [16] X. Dong, Y. Chen, X. Yin, and C. Zhuo, "Worst-case dynamic power distribution network noise prediction using convolutional neural network," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022.
- [17] Y. Chen, H. Pei, X. Dong, Z. Jin, and C. Zhuo, "Application of deep learning in back-end simulation: Challenges and opportunities," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022.
- [18] X. Dong, Y. Chen, J. Chen, Y. Wang, J. Li, T. Ni, Z. Shi, X. Yin, and C. Zhuo, "Worst-case power integrity prediction using convolutional neural network," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 4, pp. 1–19, 2023.
- [19] B. Li, S. Wang, T. Chen, Q. Sun, and C. Zhuo, "Efficient subgraph matching framework for fast subcircuit identification," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, 2024.
- [20] A. Teliti, "Graph neural networks for topology recognition of ams integrated circuits," Ph.D. dissertation, Politecnico di Torino, 2023.
- [21] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognition*, vol. 110, p. 107637, 2021.
- [22] J. R. Ullmann, "An algorithm for subgraph isomorphism," *Journal of the ACM (JACM)*, vol. 23, no. 1, pp. 31–42, 1976.
- [23] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, "A (sub) graph isomorphism algorithm for matching large graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 26, no. 10, pp. 1367–1372, 2004.
- [24] V. Carletti, P. Foggia, A. Saggese, and M. Vento, "Challenging the time complexity of exact subgraph isomorphism for huge and dense graphs with vf3," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 4, pp. 804–818, 2017.
- [25] B. Bhattacharai, H. Liu, and H. H. Huang, "Ceci: Compact embedding cluster index for scalable subgraph matching," in *Proceedings of the 2019 International Conference on Management of Data*, 2019.
- [26] A. Jüttner and P. Madarasi, "Vf2++—an improved subgraph isomorphism algorithm," *Discrete Applied Mathematics*, vol. 242, pp. 69–81, 2018.
- [27] Z. Lou, J. You, C. Wen, A. Canedo, J. Leskovec *et al.*, "Neural subgraph matching," *arXiv preprint arXiv:2007.03092*, 2020.
- [28] X. Liu, L. Zhang, J. Sun, Y. Yang, and H. Yang, "D2match: leveraging deep learning and degeneracy for subgraph matching," in *International Conference on Machine Learning*, 2023.
- [29] "NetworkX," <https://networkx.org/>.
- [30] "HyperNetX," <https://github.com/pnnl/HyperNetX>.
- [31] L. Zeng, Y. Jiang, W. Lu, and L. Zou, "Deep analysis on subgraph isomorphism," *arXiv preprint arXiv:2012.06802*, 2020.
- [32] "VF3Py," <https://pypi.org/project/vf3py/>.